

1. 数据准备

1.1 数据集构建

- 数据来源：CIFAR-10 (32×32×3 RGB图像)
- 类别选择：Cat, Dog, Bird, Deer, Horse (共5类)
- 数据预处理：
 - 灰度化：RGB (32×32×3) → Grayscale (32×32×1) 【注：后续RGB实验保留3通道】
 - 归一化：像素值 [0, 255] → [0, 1]
- 数据划分：分层随机抽样

数据集	每类样本数	总样本数
训练集	5,000	25,000
测试集	1,000	5,000

1.2 基础参数设置

参数项	设置值	备注
Batch Size	32 / 64	阶段二调整为64
Epochs	100	加入早停机制 (patience=20)
Optimizer	Adam	lr=0.001, weight_decay=1e-4
Loss Function	Cross Entropy Loss	
Activation	ReLU	
FC Neurons	512	隐藏全连接层
Random Seed	Fixed	保证可复现

2. 实验设计与结果分析

2.1 阶段一：网络深度对性能的影响

目的：固定卷积核数量为32，探究卷积池化层数量（1/2/3层）对特征提取能力的影响。

基础卷积参数：Kernel 3×3, Stride 1, Padding 1; Pooling 2×2, Stride 2。

实验结果

模型配置	Best Test Acc	Test Loss
Model-1 (1 Conv)	69.06%	0.8341
Model-2 (2 Conv)	75.58%	0.7433
Model-3 (3 Conv)	77.18%	0.6593

2.2 阶段二：通道容量与正则化优化

固定3层结构

将自变量切换为“通道数/容量”

并引入防过拟合机制（BN + Dropout + Weight Decay + BatchSize=64）

对比通道配置：

- 1. 3×64 / 3×128 / 3×256 (等宽网络)
- 2. 32-64-128 / 64-128-256 (递增型网络)

实验结果汇总

通道配置	Best Test Acc	Test Loss
3×64	81.40%	0.5392
3×128	83.80%	0.4708
32-64-128	82.46%	0.4936
64-128-256	84.42%	0.4473
3×256	84.50%	0.4502

保留两组配置：“64-128-256”+“3×256”

2.3 阶段三：注意力机制与数据增强

核心改进：

- 1. 从灰度通道恢复RGB通道
- 2. **CBAM**：通道+空间双重注意力，聚焦关键部位（耳、眼、鼻）。
- 3. **去末端池化**：特征图 4×4 → 8×8，保留4倍空间分辨率。
- 4. **数据增强**：随机遮挡迫使模型学习全局鲁棒特征。

最终优化结果

模型配置	Best Test Acc	Test Loss
64-128-256 + CBAM	86.96%	0.3929
3×256 + CBAM	87.30%	0.3847

最佳模型 (3×256 + CBAM) 详细分类指标

类别	Precision	Recall	F1-Score	Support
Bird	0.8751	0.8970	0.8859	1000
Cat	0.8037	0.8310	0.8171	1000
Deer	0.9015	0.8790	0.8901	1000
Dog	0.8861	0.8170	0.8502	1000
Horse	0.9013	0.9410	0.9207	1000
Accuracy			0.8730	5000
Macro Avg	0.8736	0.8730	0.8728	5000

3. 实验总结

阶段一：网络深度探索

1. 核心参数设置

参数项	设置值	说明
输入通道	1 (Grayscale)	RGB转灰度，降低计算量，控制变量
卷积核大小	3×3	所有层统一
卷积步长/填充	Stride=1, Padding=1	保持特征图空间尺寸不变
池化层	MaxPool 2×2, Stride=2	每层卷积后下采样一半
通道数	32 (固定)	所有卷积层输出通道均为32
全连接层	512神经元	单层隐藏FC
Batch Size	32	基础设置
优化器	Adam (lr=0.001)	无Weight Decay
训练轮数	100	Early Stopping patience=20

2. 引入机制

- **可变层数架构**: 通过参数 `num_layers` 控制1/2/3层卷积堆叠，共用同一个模型类，避免代码重复。
- **Early Stopping**: 监控验证集Loss, `patience=20`, 防止浅层模型欠拟合时过早停止、深层模型过拟合时无效训练。
- **分层数据加载**: 使用 `SubsetRandomSampler` 实现严格的分层随机抽样，确保每个batch中5类样本比例一致。

阶段二：通道容量与正则化优化

本阶段固定3层结构，将自变量切换为通道数，并系统性引入防过拟合机制。

1. 核心参数设置

参数项	设置值	变化说明
输入通道	1 (Grayscale)	仍为灰度，与阶段一保持一致
卷积层数	3 (固定)	采用阶段一最优深度
通道配置	64/128/256 或 递增	自变量 : 5种配置对比
Batch Size	64	从32调大，稳定BN统计量
Weight Decay	1e-4	新增L2正则化
Dropout	FC前0.5 + FC隐层后0.3	新增双重Dropout
其余参数	同阶段一	Conv3×3, Pool2×2, Adam lr=0.001

2. 引入机制

- **BatchNorm2d**: 每个Conv层后紧跟BN，减少内部协变量偏移，加速收敛并提供轻微正则化。
- 双重Dropout策略:
 - FC层前 `p=0.5`: 强制网络学习冗余特征，避免对特定卷积特征过度依赖。
 - FC隐层后 `p=0.3`: 防止512神经元的全连接层成为过拟合瓶颈。
- **L2正则化 (Weight Decay)**: Adam优化器中加入 `weight_decay=1e-4`，惩罚过大权重，平滑决策边界。
- **统一参数化模型类**: `CNN3Layer(channels)` 接受通道列表 `[c1, c2, c3]`，一个类支持等宽和递增两种配置，消除多模型代码差异带来的实验误差。
- **Batch Size调整**: 从32→64，使BN的均值/方差估计更准确，梯度噪声更小，训练更稳定。

阶段三：注意力机制与高级数据增强（对应 CNN4猫狗.py）

本阶段在阶段二最优配置基础上，引入CBAM、恢复RGB、强化数据增强，

1. 核心参数设置

参数项	设置值	变化说明
输入通道	3 (RGB)	恢复三通道
通道配置	3×256 / 64-128-256	沿用阶段二最优两种配置
末端池化	移除	第3层Conv后不再MaxPool，特征图8×8
数据增强	Cutout + RandomErasing	新增增强
Normalize均值/标准差	CIFAR-10 RGB统计值	从灰度参数切换为RGB参数
CBAM缩减比	r=16	通道注意力压缩比
其余正则化	BN+Dropout+WD	继承阶段二全部防过拟合手段

2. 引入机制

- CBAM 注意力模块：
 - 通道注意力**：全局平均池化+最大池化 → 共享MLP → Sigmoid，学习"哪些特征通道重要"。
 - 空间注意力**：通道维度Avg+Max → 7×7 Conv → Sigmoid，学习"特征图中哪些位置重要"。
 - 嵌入位置**：每个Conv-BN-ReLU块之后，即 Conv→BN→ReLU→CBAM。
- 去末端池化**：第3层卷积后直接接自适应池化或展平，保留8×8=64个空间位置的特征（而非4×4=16），为FC层提供4倍空间细节，对猫狗面部纹理判别至关重要。
- 强数据增强组合：
 - RandomHorizontalFlip(p=0.5)：水平翻转，增加姿态多样性。
 - RandomErasing(p=0.5, scale=(0.02,0.33))：随机擦除矩形区域，强迫模型不依赖单一局部特征。
- RGB归一化参数切换**：使用CIFAR-10官方RGB均值 (0.4914,0.4822,0.4465) 和标准差 (0.2470,0.2435,0.2616)，确保预训练先验兼容性和训练稳定性。

3. 机制协同效应说明

三通道 × CBAM × 强增强 × 去池化 四者形成正向循环：

RGB为CBAM提供颜色语义通道；

CBAM引导模型关注被Cutout遮挡后仍需识别的关键区域；

去池化保留的空间细节使CBAM的空间注意力有足够分辨率定位面部特征；

强增强迫使CBAM不能只记住固定模式，必须学习鲁棒的注意力分布。

三阶段

维度	阶段一	阶段二	阶段三
自变量	层数 (1/2/3)	通道数 (5种)	注意力+增强
输入通道	1 (Gray)	1 (Gray)	3 (RGB)
卷积层数	变	3 (固定)	3 (固定)
BatchNorm	✗	✓	✓
Dropout	✗	✓ (0.5+0.3)	✓ (0.5+0.3)
Weight Decay	✗	✓ (1e-4)	✓ (1e-4)
Batch Size	32	64	64
数据增强	✗	✗	✓ Flip+Erasing
注意力机制	✗	✗	✓ CBAM
末端池化	✓	✓	✗ 移除
对应代码	CNN1→CNN2	CNN3	CNN4